

INSTITUTO FEDERAL DE EDUCACAO, CIENCIA E TECNOLOGIA DO TOCANTINS
CAMPUS PARAISO DO TOCANTINS
CURSO DE BACHARELADO EM SISTEMAS DE INFORMACAO

DOCUMENTO DE ESPECIFICACAO:
T.O.D.A.N (TRUTH OR DARE ASYNC NETWORK)

Disciplina: Linguagens e Tecnicas de Programacao IV

Professor: Marcos Raimundo Mendes Ramos

Equipe de Desenvolvimento:

- Pedro Roberto Ribeiro Bandeira Labre

Paraiso do Tocantins - 2026

1. HISTORICO DE ALTERACOES

Data	Versao	Descricao da Alteracao	Autor
24/03/2026	1.0	Inicio da fase de prototipacao visual (light mode) e definicao inicial do escopo do aplicativo.	Pedro Roberto Ribeiro Bandeira Labre
25/03/2026	1.1	Evolucao dos prototipos light mode e organizacao dos fluxos no README para documentacao visual.	Pedro Roberto Ribeiro Bandeira Labre
26/03/2026	1.2	Conclusao da rodada de prototipos dark mode (busca, perfil, clubes, grupos, respostas e configuracoes).	Pedro Roberto Ribeiro Bandeira Labre
28/03/2026	1.3	Inicializacao tecnica do backend (Express, Prisma, PostgreSQL) e implementacao da autenticao com testes automatizados.	Pedro Roberto Ribeiro Bandeira Labre
28/03/2026	1.4	Inicializacao da base do app mobile com tela de login e implementacao da tela de cadastro integrada ao fluxo de autenticao.	Pedro Roberto Ribeiro Bandeira Labre
29/03/2026	1.5	Integracao da autenticao mobile com backend e consolidacao dos testes de frontend para login e cadastro.	Pedro Roberto Ribeiro Bandeira Labre
30/03/2026	1.6	Implementacao da tela de feed mobile e configuracao de integracao da API por variaveis de ambiente.	Pedro Roberto Ribeiro Bandeira Labre
01/04/2026	1.7	Implementacao do backend do feed com testes e isolamento do banco de testes para execucoes automatizadas.	Pedro Roberto Ribeiro Bandeira Labre
01/04/2026	1.8	Entrega de criacao de truths e dares no backend, com cobertura de testes e integracao com feed.	Pedro Roberto Ribeiro Bandeira Labre
01/04/2026	1.9	Implementacao da tela create-challenge no mobile com consumo real de usuarios, truths e dares.	Pedro Roberto Ribeiro Bandeira Labre
04/04/2026	1.10	Aprimoramentos do feed: persistencia de usuario-alvo, configuracoes de dare e regras de permissao para delete.	Pedro Roberto Ribeiro Bandeira Labre
05/04/2026	1.11	Integracao de status real de dares e likes persistidos no feed, com melhorias de comportamento de interface.	Pedro Roberto Ribeiro Bandeira Labre
06/04/2026	1.12	Implementacao da tela de comentarios do feed no mobile, com estrutura de fluxo pronta para evolucao backend.	Pedro Roberto Ribeiro

Data	Versao	Descricao da Alteracao	Autor
			Bandeira Labre
07/04/2026	1.13	Implementacao da base de perfil, configuracoes e notificacoes no mobile com tema global compartilhado.	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.14	Consolidacao das telas de busca, clubes e criacao de grupo; integracao de perfil real (username/bio) e logout funcional.	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.15	Atualizacao da documentacao com analise tecnica detalhada de backend e mobile, incluindo escopo implementado e integracoes pendentes.	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.16	Ajuste do build de producao do backend para viabilizar deploy online no Render com banco Supabase (commit 0e16e75).	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.17	Correcao dos imports do Prisma Client no artefato compilado para estabilidade do backend online no Render (commit 17cbd76).	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.18	Ajuste do prisma.config para migrations com DIRECT_URL do Supabase, garantindo fluxo de deploy (commit a32427f).	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.19	Correcao do entypoint de start em producao para manter a API online no Render (commit 4a6ffb1).	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.20	Consolidacao da publicacao do backend em nuvem: aplicacao no Render e banco PostgreSQL gerenciado no Supabase.	Pedro Roberto Ribeiro Bandeira Labre
08/04/2026	1.20.1	Terceira entrega correspondente a etapa final da primeira versao do aplicativo.	Pedro Roberto Ribeiro Bandeira Labre
09/04/2026	1.21	Implementacao da interface completa do fluxo de recuperacao de senha no mobile.	Pedro Roberto Ribeiro Bandeira Labre
10/04/2026	1.22	Ajuste de SSL do Prisma runtime para conexao com Supabase.	Pedro Roberto Ribeiro Bandeira Labre
04/05/2026	1.23	Implementacao do backend de proofs e base de envio de midias, com estrutura inicial de proof detail no mobile.	Pedro Roberto Ribeiro Bandeira Labre
05/05/2026	1.24	Implementacao da visualizacao local de proofs no mobile.	Pedro Roberto Ribeiro Bandeira Labre

Data	Versao	Descricao da Alteracao	Autor
06/05/2026	1.25	Implementacao de comentarios reais em truths.	Pedro Roberto Ribeiro Bandeira Labre
08/05/2026	1.26	Aprimoramento de comentarios e denuncias de truths, com acoes e cobertura de testes.	Pedro Roberto Ribeiro Bandeira Labre
09/05/2026	1.27	Implementacao do backend de clubes (schema, CRUD, membros, convites e solicitacoes).	Pedro Roberto Ribeiro Bandeira Labre
10/05/2026	1.28	Administracao de membros e auditoria de clubes, com publicacao de prompts.	Pedro Roberto Ribeiro Bandeira Labre
11/05/2026	1.29	Prompts de clubes (detalhe, edicao, moderacao, respostas, comentarios, likes) e feed interno.	Pedro Roberto Ribeiro Bandeira Labre
12/05/2026	1.30	Implementacao de feed agregado de clubes.	Pedro Roberto Ribeiro Bandeira Labre
13/05/2026	1.31	Reformulacao dos requisitos que ja foram implementados e dos requisitos futuros.	Pedro Roberto Ribeiro Bandeira Labre
15/05/2026	1.32	Backend: Projecção de clubes no feed geral com contadores reais de membros/posts, e ordenação cronológica do feed interno de clubes.	Pedro Roberto Ribeiro Bandeira Labre
16/05/2026	1.33	Backend: Reorganização dos serviços por domínio e cobertura final de testes de interações de clubes.	Pedro Roberto Ribeiro Bandeira Labre
17/05/2026 a 18/05/2026	1.34	Backend: Reorganização de rotas e controllers por domínio. Mobile: Criação de mappers, contratos de API para clubes (Meus Clubes e Descobrir), refresh, retry e busca remota.	Pedro Roberto Ribeiro Bandeira Labre
19/05/2026 a 20/05/2026	1.35	Backend: Implementação do serviço de e-mail e persistência de tokens para recuperação de senha. Mobile: Tela de criação de clubes com busca de membros, regras de privacidade, tags e validação de identidade.	Pedro Roberto Ribeiro Bandeira Labre
21/05/2026 a 22/05/2026	1.36	Backend: Exposição das rotas de recuperação de senha e reforço de segurança do token de reset. Mobile: Tela de detalhes de clubes estruturada com abas de feed, membros, respostas e envio de provas.	Pedro Roberto Ribeiro Bandeira Labre

Data	Versao	Descricao da Alteracao	Autor
23/05/2026 a 25/05/2026	1.37	Backend: Implementação de denúncias e notificações de clubes. Mobile: Moderação simples de clubes na UI, integração de denúncias, feed de atividades no feed geral e refatoração de Ranking para Mídias.	Pedro Roberto Ribeiro Bandeira Labre
26/05/2026 a 27/05/2026	1.38	Backend: Testes de cobertura backend da recuperação de senha. Mobile: Inbox única de notificações (badge de não lidas, leitura em lote, navegação segura) e integração visual completa do fluxo de recuperação de senha com UX aprimorada.	Pedro Roberto Ribeiro Bandeira Labre
29/05/2026 a 30/05/2026	1.39	Backend: Endpoints, mappers e serviços de busca unificada de usuários, clubes e conteúdos com testes de regressão. Mobile: Tela de busca integrada com sugestões, filtros avançados, buscas recentes locais (AsyncStorage) e debounce nas requisições.	Pedro Roberto Ribeiro Bandeira Labre
01/06/2026	1.40	Backend: Rotas de alteração de e-mail/senha e suporte/denúncias via e-mail. Mobile: Tela de configurações integrada com a API, validação de campos, sincronização de tema global e persistência local de tema.	Pedro Roberto Ribeiro Bandeira Labre
02/06/2026 a 03/06/2026	1.41	Backend: Sincronização de preferências, exclusão de conta, gerenciamento de sessões, logs seguros (safe logger) e rotina de retenção de notificações. Mobile: UI para revogação remota de logins e acessibilidade.	Pedro Roberto Ribeiro Bandeira Labre
06/06/2026	1.42	Backend: Contratos de upload de mídias, serviço de assinatura de URLs para o Supabase Storage e validação de permissões por caso de uso.	Pedro Roberto Ribeiro Bandeira Labre
08/06/2026 a 09/06/2026	1.43	Backend: Integração final do Brevo SMTP para e-mails de recuperação de senha, chamados e suporte. Mobile: Renderização de avatares, capas e mídias de provas com cache local resiliente no AsyncStorage.	Pedro Roberto Ribeiro Bandeira Labre

2. DEFINICAO DO PROJETO

2.1 Tema

Rede social assincrona baseada em Verdade ou Desafio, com interacoes por prompts de verdade/desafio, feed social e dinamicas em grupos (clubs).

2.2 Objetivo do Aplicativo

O aplicativo busca oferecer uma experiencia social de entretenimento em que usuarios criam e respondem desafios de forma assincrona, sem depender de todos estarem conectados ao mesmo tempo. O foco e

estimular engajamento por meio de perfis, feed, curtidas e participacao em grupos.

3. REQUISITOS DO SISTEMA

3.1 Requisitos Funcionais (RF)

Descrevem as acoes que o sistema deve ser capaz de executar.

- **[FEITO]** RF001: Acessar login: o app mobile deve exibir a tela de login.
- **[FEITO]** RF002: Acessar cadastro: o app mobile deve exibir a tela de cadastro.
- **[FEITO]** RF003: Cadastrar usuario: o sistema deve permitir cadastro de usuario por email e senha.
- **[FEITO]** RF004: Autenticar usuario: o sistema deve permitir autenticacao de usuario por email e senha e emitir token JWT no login autenticado.
- **[FEITO]** RF005: Armazenar token: o app mobile deve persistir o token de autenticacao localmente.
- **[FEITO]** RF006: Autorizar requisicoes: o app mobile deve enviar o token nas requisicoes protegidas.
- **[FEITO]** RF007: Consultar feed: o app mobile deve exibir a tela de feed com itens recentes.
- **[FEITO]** RF008: Consultar comentarios: o app mobile deve exibir a tela de comentarios do feed.
- **[FEITO]** RF009: Curtir truth: o sistema deve permitir curtir e descurtir truths.
- **[FEITO]** RF010: Curtir dare: o sistema deve permitir curtir e descurtir dares.
- **[FEITO]** RF011: Curtir club: o sistema deve permitir curtir e descurtir clubs.
- **[FEITO]** RF012: Excluir verdade: o sistema deve permitir excluir uma verdade quando houver permissao.
- **[FEITO]** RF013: Excluir desafio: o sistema deve permitir excluir um desafio quando houver permissao.
- **[FEITO]** RF014: Iniciar criacao de desafio: o app mobile deve exibir a tela de criacao de desafio.
- **[FEITO]** RF015: Listar usuarios: o sistema deve listar usuarios para interacao no fluxo de criacao de desafio.
- **[FEITO]** RF016: Criar verdade: o sistema deve permitir criar prompts do tipo verdade para outro usuario.
- **[FEITO]** RF017: Criar desafio: o sistema deve permitir criar prompts do tipo desafio para outro usuario.
- **[FEITO]** RF018: Acessar perfil: o app mobile deve exibir a tela de perfil.
- **[FEITO]** RF019: Consultar perfil: o app mobile deve carregar dados do proprio perfil pela API.
- **[FEITO]** RF020: Atualizar perfil: o app mobile deve atualizar dados do proprio perfil pela API.
- **[FEITO]** RF021: Acessar configuracoes: o app mobile deve exibir a tela de configuracoes.
- **[FEITO]** RF022: Encerrar sessao: o app mobile deve realizar logout removendo token local.
- **[FEITO]** RF023: Redirecionar login: o app mobile deve redirecionar para login apos logout.
- **[FEITO]** RF024: Acessar busca: o app mobile deve exibir a tela de busca.
- **[FEITO]** RF025: Acessar clubes: o app mobile deve exibir a tela de clubes.
- **[FEITO]** RF026: Acessar criacao de grupo: o app mobile deve exibir a tela de criacao de grupo.

3.2 Requisitos Nao Funcionais (RNF)

Descrevem caracteristicas de qualidade, desempenho e restricoes.

[FEITO] Comunicacao: A comunicacao entre o app e o servidor (API) deve ocorrer via protocolo web seguro e padronizado (HTTP). Os dados devem ser trocados em formato estruturado e consistente (JSON).

[FEITO] Seguranca: Requisicoes protegidas devem exigir autenticao por token (JWT). Senhas devem ser armazenadas com hash seguro (bcrypt) para reduzir riscos de vazamento.

[FEITO] Backend: O backend deve ser implementado em TypeScript para padronizar tipagem e manutencao. O backend deve compilar via TypeScript compiler (tsc) para gerar o build de producao.

[FEITO] Testes automatizados: O backend deve possuir testes automatizados com Jest para validar regras e rotas. O mobile deve possuir testes automatizados com Jest para validar telas e fluxos.

[FEITO] Banco de dados: A persistencia deve utilizar PostgreSQL como banco relacional. O acesso ao banco deve utilizar Prisma ORM para mapear modelos e consultas.

[FEITO] Versionamento: O codigo-fonte deve ser versionado com Git para historico e rastreio. O repositorio deve ser mantido no GitHub para colaboracao.

[FEITO] Navegacao mobile: O mobile deve utilizar Expo Router para navegacao baseada em rotas. O mobile deve manter stack centralizada de rotas para controle do fluxo.

[FEITO] Temas do mobile: O mobile deve suportar modo automatico (system) seguindo o sistema, modo claro (light) e modo escuro (dark) para acessibilidade.

[FEITO] Arquitetura mobile: O token de autenticao no mobile deve ser persistido com AsyncStorage para manter a sessao. O consumo de API no mobile deve ser centralizado em camada de servico para consistencia. O consumo de API no mobile deve aplicar tratamento padrao de erro para mensagens previsiveis. A arquitetura mobile deve separar componentes reutilizaveis, hooks de estado e fluxo, constantes de tema e UI e tipos compartilhados.

[FEITO] Deploy do backend: O build de producao do backend deve executar prisma generate antes da compilacao TypeScript para gerar o cliente. O backend em producao deve iniciar pelo entrypoint compilado dist/src/server.js para garantir o runtime correto. As migrations do backend devem usar conexao direta via variavel DIRECT_URL. O runtime do backend deve conectar no banco pela variavel DATABASE_URL. O servidor backend deve fazer bind em 0.0.0.0 com porta via variavel PORT para atender o ambiente de deploy. O backend deve estar publicado em ambiente de producao no Render. O banco de producao deve utilizar PostgreSQL gerenciado no Supabase. O deploy de producao deve aplicar migrations com prisma migrate deploy antes do start da API.

[FEITO] Variaveis de ambiente: O backend deve exigir as variaveis DATABASE_URL, DIRECT_URL e JWT_SECRET no ambiente de producao. O mobile deve exigir a variavel EXPO_PUBLIC_API_URL para consumo da API publicada.

3.3 Requisitos Futuros

Requisitos funcionais planejados a partir do estado atual do sistema, cobrindo clubes (CRUD, membros, convites e auditoria), feed de clubes, prompts, comentarios, provas, uploads e funcionalidades mobile (recuperacao de senha, busca, notificacoes e configuracoes).

- **[FEITO] RFF01:** Gerenciar clube: o sistema deve permitir criar, atualizar, consultar detalhes, arquivar e restaurar clubes.
- **[FEITO] RFF02:** Descobrir clubes: o sistema deve permitir listar meus clubes, descobrir clubes disponiveis e buscar clubes por criterio.
- **[FEITO] RFF03:** Gerenciar membros: o sistema deve permitir listar membros com filtros e busca, alterar papel, remover membro e transferir propriedade.
- **[FEITO] RFF04:** Gerenciar entradas: o sistema deve permitir criar convites, listar convites, aceitar/recusar convites, entrar em clubes publicos, solicitar entrada em clubes privados, aprovar/rejeitar solicitacoes e sair do clube.

- **[FEITO]** RFF05: Notificacoes de clube: o sistema deve permitir silenciar e desilenciar notificacoes do clube.
- **[FEITO]** RFF06: Auditar acoes: o sistema deve registrar e permitir consultar auditoria de acoes de membros.
- **[FEITO]** RFF07: Feed de clubes: o sistema deve disponibilizar feed agregado e feed do clube.
- **[FEITO]** RFF08: Gerenciar prompts: o sistema deve permitir criar/publicar, editar, detalhar, moderar e definir campos e permissoes de prompts de clube.
- **[FEITO]** RFF09: Interagir com prompts: o sistema deve permitir curtir prompt, responder prompt, listar respostas, curtir resposta e comentar prompt de clube.
- **[FEITO]** RFF10: Comentarios de truths: o sistema deve permitir listar, criar, curtir e responder comentarios de truths.
- **[FEITO]** RFF11: Enviar prova: o sistema deve permitir enviar prova de desafio com midia e texto opcional.
- **[FEITO]** RFF12: Gerar upload: o sistema deve permitir gerar URL assinada para upload de midia.
- **[FEITO]** RFF13: Visualizar prova: o app mobile deve disponibilizar tela de detalhes da prova do desafio.
- **[FEITO]** RFF14: Solicitar codigo: o app mobile deve permitir solicitar codigo de recuperacao de senha.
- **[FEITO]** RFF15: Reenviar codigo: o app mobile deve permitir reenviar codigo de recuperacao de senha.
- **[FEITO]** RFF16: Validar codigo: o app mobile deve permitir validar codigo de recuperacao de senha.
- **[FEITO]** RFF17: Redefinir senha: o app mobile deve permitir redefinir senha.
- **[FEITO]** RFF18: Buscar usuarios e clubes: o app mobile deve permitir buscar usuarios e clubes por termo e visualizar resultados.
- **[FEITO]** RFF19: Exibir sugestoes: o app mobile deve exibir usuarios recomendados e clubes em alta no estado inicial da busca.
- **[FEITO]** RFF20: Gerenciar recentes: o app mobile deve permitir salvar, remover e limpar buscas recentes.
- **[FEITO]** RFF21: Abrir resultado: o app mobile deve permitir abrir perfil publico de usuario ou detalhe de clube a partir da busca.
- **[FEITO]** RFF22: Listar notificacoes: o app mobile deve permitir listar notificacoes do usuario.
- **[FEITO]** RFF23: Marcar notificacao: o app mobile deve permitir marcar notificacao como lida.
- **[FEITO]** RFF24: Marcar todas: o app mobile deve permitir marcar todas as notificacoes como lidas.
- **[FEITO]** RFF25: Exibir nao lidas: o app mobile deve exibir a contagem de notificacoes nao lidas.
- **[FEITO]** RFF26: Abrir notificacao: o app mobile deve permitir navegar para o conteudo relacionado a notificacao.
- **[FEITO]** RFF27: Carregar dados: o app mobile deve carregar dados reais do usuario na tela de configuracoes.
- **[FEITO]** RFF28: Alterar privacidade: o app mobile deve permitir alternar conta privada/publica.
- **[FEITO]** RFF29: Alterar email: o app mobile deve permitir alterar o email da conta.
- **[FEITO]** RFF30: Alterar senha: o app mobile deve permitir alterar a senha da conta.
- **[FEITO]** RFF31: Persistir tema: o app mobile deve persistir a preferencia de tema entre sessoes.
- **[FEITO]** RFF32: Solicitar suporte: o app mobile deve permitir enviar denuncia de abuso ou contato com suporte.

- **[FEITO] RFF33:** Excluir conta: o app mobile deve permitir excluir a propria conta.

3.4 Requisitos Futuros Nao Funcionais

Requisitos de qualidade e restricoes planejados para evolucoes futuras do sistema.

[FEITO] Desempenho: Buscas, notificacoes e feeds de clubes devem responder com latencia consistente, usando paginacao (cursor/limit) e payloads enxutos.

[FEITO] Escalabilidade: O backend deve suportar crescimento de clubes, prompts e notificacoes com indices adequados e limites por requisicao.

[FEITO] Seguranca: O fluxo de recuperacao de senha deve usar tokens com expiracao, hash seguro (bcrypt/SHA-256), escopo de reset e limitacao de tentativas.

[FEITO] Privacidade: A busca deve respeitar contas privadas e visibilidade de clubes, evitando exposicao de dados sensiveis em resultados ou notificacoes.

[FEITO] Confiabilidade: A criacao de notificacoes deve ser idempotente (deduplicacao) para evitar repeticao por eventos iguais.

[FEITO] Disponibilidade: Falhas em servicos externos (ex.: envio de e-mail) nao devem interromper o fluxo do usuario; o sistema deve degradar com retorno generico e log interno.

[FEITO] Consistencia: Operacoes de clubes (membros, convites, prompts) devem manter contadores e audit log consistentes, preferindo transacoes.

[FEITO] Retencao: Notificacoes devem seguir politica de retencao (ex.: 90 dias) e permitir limpeza segura sem perda de integridade.

[FEITO] Experiencia: A busca deve aplicar debounce e cancelamento de requisicoes para evitar chamadas excessivas e travamentos na UI.

[FEITO] Persistencia local: Preferencias do app (tema, estado de filtros) devem ser armazenadas em AsyncStorage e restauradas na inicializacao.

[FEITO] Observabilidade: O sistema deve registrar logs estruturados e metricas basicas para recuperacao de senha, notificacoes e busca, sem registrar dados sensiveis.

[FEITO] Acessibilidade: As telas futuras devem manter contraste adequado, rotulos de acessibilidade e alvos de toque coerentes.

3.5 Requisitos não programados

Funcionalidades e requisitos de qualidade que foram implementados no codebase do projeto (backend e mobile) porém não constavam no escopo e no planejamento inicial das entregas.

3.5.1 Requisitos Funcionais não Programados (RFNP)

- **Gerenciamento de Sessões Ativas (Segurança):** O app exibe os logins ativos do usuário identificando o dispositivo e a plataforma (Android/iOS/Web), permitindo a revogação de sessões específicas ou a desconexão de todos os outros aparelhos remotamente.
- **Histórico de Buscas Recentes:** O aplicativo armazena as últimas pesquisas do usuário localmente (AsyncStorage) e disponibiliza uma interface para removê-las individualmente ou limpar todo o histórico de busca.
- **Sincronização de Preferências:** Persistência remota e local das preferências do usuário (como o estado da conta privada, temas de interface e filtros de busca definidos) sincronizada com o banco de dados.

- **Denúncia de Comentários de Desafios:** Extensão da moderação permitindo que o usuário denuncie comentários inadequados ou abusivos diretamente na aba de comentários do feed.
- **Sistema de Chamados de Suporte e Denúncias de Abuso (Ticketing):** O sistema permite criar chamados de suporte técnico e denúncias de abuso (com categorias como spam, ódio, violência, nudez e outros) que são registrados no banco de dados e notificados por e-mail para a moderação.
- **Limitação Dinâmica para Ações de Clubes (DB-Backed Rate Limiting):** O backend restringe a criação excessiva de entidades críticas de clubes (limite de 5 novos clubes por hora, 20 convites por hora e 30 prompts de clube por hora) consultando diretamente os registros no banco de dados e lançando exceções amigáveis.
- **Normalização de Slugs Únicos para Clubes:** Geração automática de caminhos/slugs de URL limpos, normalizados (sem acentuação ou caracteres especiais) e únicos com resolução de colisões até 1000 sufixos.
- **Moderação Avançada e Restrição de Membros de Clubes:** Sistema que permite a administradores ou donos bloquearem membros (removendo-os do contador de membros ativos do clube) e suspenderem permissões de postagem temporariamente até uma data futura específica, persistindo as ações no registro de auditoria do clube.

3.5.2 Requisitos Não Funcionais não Programados (RNFNP)

- **Endpoint de Diagnóstico (Health Check):** O backend expõe o endpoint público `/app-info` para checagem rápida de saúde (health check), retornando o status do sistema, a versão ativa do pacote e o ambiente de deploy.
- **Observabilidade e Safe Logger:** Sistema de auditoria de logs no backend que mascara dados pessoais e sensíveis de usuários (e-mails, senhas, chaves secretas) nos arquivos de registro de depuração.
- **Política de Retenção de Notificações:** Execução de rotina periódica no backend para apagar permanentemente notificações com mais de 90 dias para otimização da base de dados.
- **Rate Limiting e Segurança de Recuperação:** Expiração cronometrada de tokens e limitação estrita de tentativas para validação do código de segurança de reset de senha (bloqueio automático contra força bruta).
- **Limitação de Taxa Global (Rate Limiting):** O backend possui um middleware em memória (`rate-limit.middleware.ts`) que bloqueia chamadas sucessivas vindas de um mesmo endereço IP em endpoints críticos.
- **Paginação Padronizada por Cursor e Offset:** Módulo utilitário no backend (`pagination.ts`) que padroniza os parâmetros de limite e cursor, limitando o volume de dados por requisição a um teto seguro (máximo 50 itens) para manter a performance.
- **Prevenção de Cache de Informações Sensíveis:** O mecanismo de cache local (`cache.ts`) valida payloads e impede que dados como chaves secretas, chaves de API, senhas ou tokens sejam salvos em cache no dispositivo móvel.
- **Isolamento de Cache Local por Usuário (Multi-tenant):** O cache do `AsyncStorage` separa os dados locais com namespaces baseados no ID do usuário extraído do JWT autenticado, garantindo que usuários diferentes no mesmo dispositivo não vejam dados cacheados um do outro.
- **Métricas de Alteração de Credenciais em Memória:** O backend rastreia o volume diário de trocas de e-mail e senha em memória, permitindo observar tendências de atividades suspeitas por meio de logs estruturados sem armazenar dados pessoais.
- **Validação e Permissões Unificadas de Uploads por Caso de Uso:** Camada de serviço de uploads centralizada no backend que valida o tipo de arquivo e assegura que o solicitante tenha a autoridade

devida dependendo do contexto de uso (avatar de perfil, avatar/capa de clube, anexo em comentário, anexo em prompt, resposta a prompt ou prova de desafio).

3.6 Modelagem

Modelagem de alto nível das funcionalidades e telas do aplicativo.

Modulo	Entidades/Recursos	Telas/Fluxos	Status mobile atual
Autenticação	User, JWT	login, signup-screen	Totalmente integrado com a API do backend e persistência local de token.
Recuperação de senha	PasswordResetToken	forgot-password, verify-code, reset-password, password-success	Totalmente integrado com a API de recuperação de senha do backend.
Feed	Truth, Dare, Comment	feed, feed-comments, create-challenge	Totalmente integrado com a API do backend (criação e exclusão de desafios, likes e comentários).
Perfis	User	profile, profile/[id], settings	Totalmente integrado (visualização de perfil próprio e público, edição e logout).
Provas / Envio de mídias	DareProof, Media	action-screen, proof-detail	Totalmente integrado com a API de provas e fluxo de upload de imagens.
Notificações	Notification	notifications	Totalmente integrado com a API de notificações do backend (listagem, contagem e marcação de leitura).
Interações	Like (truth/dare/club)	feed, cards de interação	Curtida e descurtida integrada com endpoints de truth, dare e club.
Clubes/Grupos	Club, ClubMember, ClubPrompt, ClubPromptResponse	clubs, clubs/[id], create-group, search	Totalmente integrado com a API do backend (criação, feed de clubes, moderação, auditoria e buscas).

4. REQUISITOS TECNICOS DO PROJETO

Definicao da arquitetura e ferramentas utilizadas.

Linguagem	TypeScript (backend), TypeScript/TSX (mobile)
Framework / Plataforma	Express 5 (backend), React Native 0.81 + Expo SDK 54 + Expo Router v6 (mobile)
Deploy Backend (estado atual)	Build de producao: prisma generate && tsc -p tsconfig.build.json. Start de producao: node dist/src/server.js. Prisma migrations: prisma.config.ts com datasource.url em env('DIRECT_URL'). Runtime Prisma: adapter PostgreSQL usando process.env.DATABASE_URL. Servidor HTTP: app.listen com host 0.0.0.0 e porta por process.env.PORT. Infra alvo: API no Render e banco PostgreSQL no Supabase.
Variaveis de Ambiente Criticas	Backend (runtime): DATABASE_URL, DIRECT_URL, JWT_SECRET, PORT, SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD, EMAIL_FROM, EMAIL_PROVIDER, BREVO_API_KEY, SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_STORAGE_BUCKET. Backend (migrations): DIRECT_URL. Mobile: EXPO_PUBLIC_API_URL. Testes backend: TEST_DATABASE_URL (via scripts/run-prisma-test-command.ts).
Runbook de Deploy (Render)	1) Build: npm run build (executa prisma generate e tsc para gerar a build de produção em dist/src). 2) Migrations: npx prisma migrate deploy (utilizando a variável DIRECT_URL para migração direta no Supabase). 3) Start: npm run start (inicia pelo entrypoint compilado dist/src/server.js). 4) Validação: requisição GET para /app-info para validar status: 'ok'.
Automacao de Entrega	Não há workflow de CI/CD versionado em .github/workflows no estado atual. Não há arquivo de infraestrutura declarativa de deploy (ex.: render.yaml) no repositório. Publicação atual é baseada na integração direta do GitHub com o Render.
Arquitetura Mobile (estado atual)	Navegação por expo-router com estrutura de abas (tabs) e stack centralizada (app/_layout.tsx) cobrindo autenticação, feed, busca, clubes, perfil, comentários, configurações, notificações, recuperação de senha e provas. Tema global por ThemeContext (system/light/dark). Camada de dados estruturada em mobile/services (api.ts, clubsApi.ts, notificationsApi.ts) com parse padrão de resposta, token de

	autenticação persistido em AsyncStorage, e sistema de cache local com expiração e invalidação (cache.ts, cachedApi.ts).
Banco de Dados	PostgreSQL (Supabase em produção / Local em desenvolvimento) com Prisma ORM
Controle de Versao	Git/GitHub
Servicos de Terceiros	Render (hospedagem da API backend), Supabase (PostgreSQL gerenciado e Storage em produção) e Brevo (envio de e-mails de segurança, chamado de suporte e recuperação de senha).
Testes Automatizados (estado documentado)	Backend: 76 arquivos de teste em backend/tests. Mobile: 60 arquivos em mobile/___tests___. Mobile cobre login, cadastro, busca, clubes, perfil, comentários, notificações, configurações da conta, recuperação de senha, cache local e hooks customizados. Relatórios em docs/backend/tests e docs/mobile/tests com suítes recentes em status PASS.
Escopo Mobile Mapeado	25 telas em app/, 141 componentes reutilizaveis, 25 hooks de estado/fluxo, 10 arquivos de constantes de tema e UI, 13 arquivos de tipos.

5. IDEIAS FUTURAS (BACKLOG)

Funcionalidades planejadas para versoes posteriores.

- **[FEITO]** Implementacao completa do fluxo de recuperacao de senha no backend e no app.
- **[FEITO]** Implementacao das funcionalidades completas de clubes (CRUD, membros, convites, entradas, auditoria e feeds).
- **[FEITO]** Implementacao dos prompts de clubes (publicacao, edicao, moderacao, respostas, comentarios e likes).
- **[FEITO]** Implementacao da busca com resultados reais, sugestoes e historico de buscas recentes.
- **[FEITO]** Implementacao do centro de notificacoes (listagem, leitura, badge e navegacao para conteudo relacionado).
- **[FEITO]** Implementacao das funcionalidades de configuracoes (privacidade, alterar email/senha, tema, suporte e exclusao de conta).
- **[FEITO]** Implementacao do fluxo de provas com upload assinado e visualizacao detalhada no app.

ANEXO A - EVIDENCIAS TECNICAS IMPLEMENTADAS

Backend - rotas principais

Base	Endpoints principais
/auth	POST /signup (cadastro de usuário) POST /login (login/autenticação) POST /password-reset/forgot-password (solicitar código de recuperação por e-mail) POST /password-reset/verify-reset-code (validar código de segurança de 6 dígitos) POST /password-reset/reset-password (redefinir para nova senha ativa)

Base	Endpoints principais
/users	GET /me (obter perfil do próprio usuário autenticado) PUT /me (atualizar nome de usuário, bio e dados básicos) PATCH /me (alterar e-mail ou senha da conta) DELETE /me (exclusão lógica/permanente da conta) GET /me/sessions (listar logins e dispositivos ativos do usuário) DELETE /me/sessions (desconectar todas as outras sessões ativas) DELETE /me/sessions/:id (desconectar um dispositivo/sessão específica) GET /me/preferences (carregar preferências de busca e privacidade) PUT /me/preferences (atualizar e salvar preferências de busca/privacidade) GET /:id/public (consultar perfil público de outro usuário) GET / (listar usuários no banco para fluxo de desafios)
/feed	GET / (obter feed geral assíncrono de verdades e desafios com paginação por cursor)
/truths	POST / (criar prompt do tipo verdade direcionado a outro usuário) DELETE /:id (excluir verdade criada por mim) POST /:id/like (curtir ou descurtir uma verdade) POST /:id/report (denunciar verdade por abuso/conteúdo inadequado) GET /:id/comments (listar comentários estruturados da verdade) POST /:id/comments (comentar em uma verdade) PATCH /comments/:id (editar comentário próprio) DELETE /comments/:id (excluir comentário próprio) POST /comments/:id/like (curtir/descurtir comentário) POST /comments/:id/report (denunciar comentário inadequado)
/dares	POST / (criar prompt do tipo desafio direcionado a outro usuário) DELETE /:id (excluir desafio criado por mim) POST /:id/like (curtir ou descurtir um desafio) POST /:id/proof (enviar prova com texto e anexo de mídia no Supabase Storage) GET /proofs/:proofId (carregar detalhes visuais e de texto de uma prova de desafio)
/clubs	POST / (criar um novo clube/grupo) GET /my (listar os clubes em que o usuário participa) GET /invites/my (listar convites de clubes pendentes para mim) POST /invites/:id/accept (aceitar convite e entrar no clube) POST /invites/:id/decline (recusar convite pendente) POST /join-requests/:id/approve (aprovar solicitação de entrada de outro usuário) POST /join-requests/:id/reject (rejeitar solicitação de entrada de outro usuário) GET /discover (listar clubes públicos disponíveis para descoberta) GET /search (buscar clubes por palavra-chave) GET /:id/members (listar membros com papéis e status) PUT /:id/members/:userId/role (atualizar papel do membro: owner, admin, member) DELETE /:id/members/:userId/remove (remover/expulsar membro do clube) POST /:id/members/:userId/block (bloquear usuário permanentemente no clube) POST /:id/members/:userId/suspend-posting (suspender postagens do membro por período determinado) POST /:id/transfer-ownership (transferir a propriedade do clube para outro membro) GET /:id/audit-logs (consultar logs de auditoria e ações de moderação do clube) POST /:id/join-requests (solicitar entrada em clube privado) POST /:id/join (entrar diretamente em clube público)

Base	Endpoints principais
	POST /:id/leave (sair do clube voluntariamente) POST /:id/mute (silenciar notificações de posts/mensagens do clube) POST /:id/unmute (reativar notificações do clube) GET /:id (obter detalhes do clube por ID) PATCH /:id (atualizar nome, descrição, avatar e capa do clube) DELETE /:id (arquivar/excluir clube por completo) POST /:id/restore (restaurar clube anteriormente desativado) POST /:id/report (denunciar clube por comportamento inadequado) POST /:id/like (curtir/descurtir clube)
/clubs (Feed e Prompts)	GET /feed (feed agregado de posts de todos os clubes participantes) GET /:id/feed (feed interno de posts/prompts de um clube específico) POST /:id/feed/seen (marcar feed do clube como visualizado) POST /:id/prompts (criar/publicar um prompt de verdade/desafio no clube) GET /:id/prompts/:promptId (obter detalhes de um prompt de clube) PATCH /:id/prompts/:promptId (editar texto e opções do prompt do clube) DELETE /:id/prompts/:promptId (moderar/apagar prompt de clube) POST /:id/prompts/:promptId/like (curtir prompt de clube) POST /:id/prompts/:promptId/report (denunciar prompt de clube) POST /:id/prompts/:promptId/responses (responder ao prompt do clube com texto/mídia) GET /:id/prompts/:promptId/responses (listar respostas publicadas para o prompt) POST /:id/prompts/:promptId/responses/:responseId/like (curtir resposta do prompt) POST /:id/prompts/:promptId/responses/:responseId/report (denunciar resposta de prompt) POST /:id/prompts/:promptId/comments (comentar em um prompt de clube) POST /:id/prompts/:promptId/comments/:commentId/report (denunciar comentário de prompt)
/search	GET / (busca global unificada por termo de texto) GET /users (pesquisar usuários por nome/username) GET /clubs (pesquisar clubes por nome/slug) GET /content (pesquisar textos de verdades/desafios) GET /recommended/users (obter sugestões de usuários recomendados) GET /trending/clubs (obter lista de clubes populares/em alta)
/notifications	GET / (listar histórico de notificações com paginação por cursor) GET /unread-count (obter contagem numérica de notificações pendentes não lidas) PATCH /:id/read (marcar notificação específica como lida) POST /read-all (marcar todas as notificações do usuário como lidas)
/uploads	POST /sign (gerar URL assinada temporária no Supabase Storage para uploads seguros de mídias de avatar, capas e provas)
/support	POST /report-abuse (receber chamados de suporte técnico ou denúncias de violações enviando e-mail automático via Brevo para moderadores)
/app-info	GET / (endpoint público de health check do status do backend, versão e ambiente)

Backend - evidencias de deploy em producao

Arquivo	Configuracao observada	Finalidade
backend/package.json	"build": "prisma generate && tsc -p tsconfig.build.json"	Gera cliente do Prisma e compila o código TypeScript em dist/src para produção.
backend/package.json	"start": "node dist/src/server.js" e "main": "dist/src/server.js"	Define o entrypoint compilado e comando de inicialização corretos no Render.
backend/tsconfig.build.json	Build dedicado incluindo src/**/*.ts e excluindo testes	Evita que arquivos de testes automatizados e depuração sejam compilados no bundle de produção.
backend/prisma.config.ts	datasource.url: env('DIRECT_URL')	Usa conexão direta para executar migrations via Prisma no banco de dados do Supabase.
backend/src/lib/prisma.ts	PrismaPg com connectionString em process.env.DATABASE_URL e SSL condicional (rejectUnauthorized: false)	Conexão em tempo de execução da API no Render ao PostgreSQL do Supabase, forçando SSL para conexões de nuvem e evitando falhas de handshake.
backend/src/server.ts	app.listen(PORT, '0.0.0.0')	Permite binding externo exigido pelas portas do ambiente de deploy do Render.
backend/src/routes/app-info/app-info.routes.ts	GET /app-info exposto sem autenticação pública	Endpoint público de health check usado pelo Render para validar o status ativo ("ok") da aplicação pós-deploy.
Ambiente de producao	Render + Supabase PostgreSQL + Supabase Storage + Brevo SMTP	Infraestrutura de nuvem ativa onde o backend está publicado e integrado para rodar a aplicação em produção.

Mobile - consumo atual de API e estado de integracao

Fluxo mobile	Funcoes/Endpoints consumidos	Status
Autenticação	signup(), login(), saveToken(), getToken()	Integrado com backend (telas: login.tsx, signup-screen.tsx)
Recuperação de senha	requestPasswordReset(), verifyResetCode(), resetPassword()	Integrado com backend (hook: useRecoveryFlow)
Feed	getFeed(), toggleLike(), deleteTruth(), deleteDare()	Integrado com backend (hooks: useFeedState, useDeleteChallenge, com fallback local em falhas)

Fluxo mobile	Funcoes/Endpoints consumidos	Status
Criar desafio	getUsers(), createTruth(), createDare()	Integrado com backend (tela: create-challenge.tsx)
Perfil	getMyProfile(), updateMyProfile(), getPublicUserProfile()	Integrado com backend (hook: useProfileScreen)
Provas / Envio de mídias	submitDareProof(), getDareProof(), getUploadUrl() (uploads/mediaPicker)	Integrado com backend (hooks: useActionScreen, useProofDetailScreen)
Configurações	getMe(), updateMe(), changeEmail(), changePassword(), deleteAccount(), getUserPreferences(), updateUserPreferences(), getUserSessions(), revokeUserSession(), revokeOtherUserSessions(), reportAbuse(), removeToken()	Integrado com backend (hook: useSettingsScreen)
Notificações	listNotifications(), getUnreadNotificationsCount(), markNotificationRead(), markAllNotificationsRead()	Integrado com backend (hooks: useNotificationsScreen, useNotificationsUnreadCount)
Busca	searchUsers(), searchClubs(), searchContent(), getRecommendedUsers(), getTrendingClubs()	Integrado com backend (hook: useSearchScreen)
Clubes	getMyClubs(), discoverClubs(), getClubDetails(), joinClub(), leaveClub(), muteClub(), unmuteClub(), inviteClubUser(), getClubMembers(), blockClubMember(), suspendClubMemberPosting(), getClubAuditLogs(), createClubPrompt(), getClubFeed(), markClubFeedSeen(), createClubPromptResponse(), reportClub()	Integrado com backend (hooks: useClubsScreen, useClubDetailsScreen, useClubFeed, useClubMembers, useClubAuditLog, useClubModeration, etc.)
Comentários	getTruthComments(), createTruthComment(), toggleTruthCommentLike(), updateTruthComment(), deleteTruthComment(), reportTruthComment(), reportTruth(), reportClubPrompt()	Integrado com backend (hook: useFeedCommentsScreen)
Criar grupo	getUsers(), createClub(), updateClub()	Integrado com backend (hook: useCreateGroupScreen)